

Especificación

Algoritmos y estructuras de datos

Elías Cerdeira



¡Bienvenidos!

Presentación

- Elías Cerdeira (JTP)
- Román Gorojovsky (JTP)
- Nicolás Sawczuk (Ay1)
- Belén Alvariñas (Ay1)
- Franco Martínez Quispe (Ay1)
- Fermín Schlottmann (Ay2)
- Antonella Berzzotti (Ay2)

¡Encantados!



¿Cómo son las clases prácticas?



Las clases son más
dinámicas si
participamos todos

Aprobación de la práctica

- La materia cuenta con **dos parciales**
 - Cada parcial tiene su recuperatorio al final del cuatrimestre
- Cada examen se aprueba con **60 y 50% de cada ejercicio**
- La materia es **promocionable** con promedio 70 o más



Fechas importantes

- **Primer parcial:** Miércoles 30/09/2026
- **Segundo parcial:** Miércoles 11/11/2026
- **Primer recuperatorio:** Miércoles 25/11/2026
- **Segundo recuperatorio:** Viernes 04/12/2026



Importante

- El material de la materia está disponible en el **campus**
 - Cada apunte, guía o resuelto puede sufrir modificaciones que serán marcadas con otro color
 - Si encuentran errores **avísennos**
- La comunicación oficial se hará vía anuncios del campus que llegan por **mail**
- Las consultas se hacen **en clase** o en el **campus**



¿Qué vamos a ver hoy?

Parte 2. Especificación

- Especificación de predicados (**pred**) y funciones auxiliares (**aux**)
- Especificación de problemas (**proc**)



¿Por qué queremos especificar problemas formalmente?

- Ayuda a entender mejor el problema
- El lenguaje natural es ambiguo
- A veces es un paso intermedio en demostraciones formales
- Expresa **QUÉ** debe cumplir una solución de un problema
- No expresa **CÓMO** solucionarlo



¡A resolver ejercicios!



Dicen que ahora
se pone
entretenido...

Pequeño repaso

Variables	Fórmula	Valor
$x = 3$	$x + 2 > 4$	TOP SECRET
$a = \perp$	$True \vee a$	
$a = \perp$	$True \vee_L a$	
$s = \langle 7, 3, 5 \rangle$	$s[2] > 0$	
$s = \langle 7, 3, 5 \rangle$	$s[4] > 0$	
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \rightarrow_L esPrimo(s[i]))$	
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \rightarrow esPrimo(s[i]))$	
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \wedge_L esPrimo(s[i]))$	
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \rightarrow_L s[i] < s[i + 1])$	
$s : seq\langle \mathbb{Z} \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < s \rightarrow_L s[i] < s[i + 1])$	
$s : seq\langle \mathbb{Z} \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < s - 1 \rightarrow_L s[i] < s[i + 1])$	

Sintaxis

Variables	Fórmula	Valor
$x = 3$	$x + 2 > 4$	<i>True</i>
$a = \perp$	$True \vee a$	$\perp$
$a = \perp$	$True \vee_L a$	<i>True</i>
$s = \langle 7, 3, 5 \rangle$	$s[2] > 0$	<i>True</i>
$s = \langle 7, 3, 5 \rangle$	$s[4] > 0$	$\perp$
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \rightarrow_L esPrimo(s[i]))$	<i>True</i>
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \rightarrow esPrimo(s[i]))$	$\perp$
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \wedge_L esPrimo(s[i]))$	<i>False</i>
$s = \langle 7, 3, 5 \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < 3 \rightarrow_L s[i] < s[i + 1])$	$\perp$
$s : seq\langle \mathbb{Z} \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < s \rightarrow_L s[i] < s[i + 1])$	$\perp$
$s : seq\langle \mathbb{Z} \rangle$	$(\forall i : \mathbb{Z}) (0 \leq i < s - 1 \rightarrow_L s[i] < s[i + 1])$	Depende



Sintaxis

```
pred SoyUnPred(a:  $\mathbb{Z}$ ) {  
    a > 10  $\vee$  a mod 2 = 1  
}
```

```
aux HagoAlgo(b:  $\mathbb{Z}$ ):  $\mathbb{Z}$  =  
    (b / 5) + 3
```

Son un reemplazo
sintáctico

IfThenElse(Condición, A, B)

- Es una auxiliar especial
- Ambos valores A y B son del mismo tipo
- Si se cumple la condición, se devuelve A
- Caso contrario, se devuelve B



Predicados y auxiliares

Escribir un ***predicado*** que sea verdadero si y sólo si un entero n **divide** a otro entero m

¿Cómo se arranca
este **pred**?



Predicados y auxiliares

Escribir un *predicado* que sea verdadero si y sólo si un entero n **divide** a otro entero m

```
pred divide( $n: \mathbb{Z}$ ,  $m: \mathbb{Z}$ ) {  
  
}
```

¿Y ahora?



Predicados y auxiliares

Escribir un *predicado* que sea verdadero si y sólo si un entero n **divide** a otro entero m

```
pred divide(n:  $\mathbb{Z}$ , m:  $\mathbb{Z}$ ) {  
     $n \neq 0 \wedge m \bmod n = 0$   
}
```

¿Qué ocurre si
no está $n \neq 0$?



¡Una vez más!



A ver con qué
nos sorprende la
vida...

Predicados

Escribir un predicado que sea verdadero sii:

1. Un entero n es **primo**
2. Un entero m es el mayor **primo** que divide a otro entero n
3. Un entero e **está** en una secuencia de enteros s
4. Un secuencia de caracteres p es **prefijo** de otra secuencia s
5. Si todos los números primos de una secuencia s de enteros están en posiciones pares
6. Si dentro de la secuencia de enteros s hay un elemento **primo** que **divide** al resto



Auxiliares

Escribir una **auxiliar** que sume 10 a un entero n

¿Cómo se arranca
este **aux**?



Auxiliares

Escribir una ***auxiliar*** que sume 10 a un entero n

```
aux sumaDiez(x:  $\mathbb{Z}$ ):  $\mathbb{Z}$  =
```

¿Y ahora?



Auxiliares

Escribir una ***auxiliar*** que sume 10 a un entero n

```
aux sumaDiez(x:  $\mathbb{Z}$ ):  $\mathbb{Z}$  = x + 10
```

Excelente

Pero esto no
termina acá...



¡Ya casi!



Hay aroma a
recreo...

Predicados y auxiliares

Escribir una auxiliar que:

1. Devuelve el mayor de dos enteros contenidos en una tupla ***t***
2. Devuelve el dígito menos significativo de un entero ***n***
3. Cuenta la cantidad de apariciones de un elemento ***e*** en una secuencia ***s***
4. Suma los valores que se encuentran en posiciones pares en una secuencia ***s***



**RECESO,
¡HUYAMOS!**



Sintaxis

```
proc Nombre(in a:  $\mathbb{Z}$ , in s: seq< $\mathbb{Z}$ >):  $\mathbb{Z}$  {  
    requiere  $\{|s| > 0\}$   
    asegura  $\{res \in s \wedge res < a\}$   
}
```

Aclaración

- **NO** se pueden utilizar *procs* dentro de otros *procs*
- **SÍ** se pueden usar *pred* y *aux* para hacer más clara la lectura y comprensión de los *procs*
- Vamos a tener una sola cláusula ***requiere*** y una sola cláusula ***asegura***



Problemas

Especifica un problema que dados dos enteros n y m decide si n es múltiplo de m



Problemas

Especifica un problema que dados dos enteros n y m decide si n es múltiplo de m

```
proc esMúltiplo(in n: ℤ, in m: ℤ): bool {  
    requiere{    }  
    asegura{  
        }  
}
```



Problemas

Especifica un problema que dados dos enteros n y m decide si n es múltiplo de m

```
proc esMúltiplo(in n:  $\mathbb{Z}$ , in m:  $\mathbb{Z}$ ): bool {  
    requiere{True}  
    asegura{  $(n \neq 0 \wedge m \bmod n = 0) \vee$   
              $(m = 0 \wedge n = 0)$  }  
}
```



Problemas

Especifica un problema que dados dos enteros n y m decide si n es múltiplo de m

```
proc esMúltiplo(in
```

BONUS TRACK

¿A alguien se le
ocurre otra manera
de hacerlo?



Problemas

Especifica un problema que dados dos enteros n y m decide si n es múltiplo de m

```
proc esMúltiplo(in n:  $\mathbb{Z}$ , in m:  $\mathbb{Z}$ ): bool {  
    requiere{True}  
    asegura{res = True  $\leftrightarrow$  ( $\exists k : \mathbb{Z}$ )(m * k = n)}  
}
```

¡Esto es aún
más raro!



¡A resolver ejercicios!



Ahora que
descansaron,
¡sigamos!

Ejercicio 13.d

factoresPrimos. Especificar un problema que dado un entero positivo n obtiene su descomposición en factores primos. El problema debe devolver una secuencia de tuplas (p, e) donde p es un factor primo y e es su exponente. La salida debe estar ordenada con respecto a p



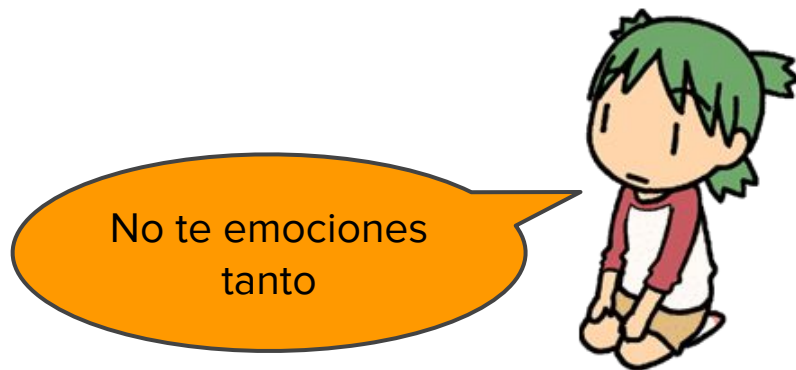
Ejercicio 13.d

```
proc factoresPrimos(in  $n : \mathbb{Z}$ ) : seq⟨tupla ⟨ $\mathbb{Z}, \mathbb{Z}$ ⟩⟩ {  
  requiere {  $n > 0$  }  
  asegura {  
     $factoriza(n, res) \wedge primerosElementosSonPrimos(res) \wedge$   
     $ordenadaEstrictamentePorPrimero(res)$   
  }  
  
  pred factoriza( $n : \mathbb{Z}, s : seq⟨tupla ⟨\mathbb{Z}, \mathbb{Z}⟩⟩$ ) {  $\prod_{i=0}^{|s|-1} (s[i]_0)^{s[i]_1} = n$  }  
  pred primerosElementosSonPrimos( $s : seq⟨tupla ⟨\mathbb{Z}, \mathbb{Z}⟩⟩$ ) {  
    ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |s| \rightarrow_L esPrimo(s[i]_0)$ ) }  
  pred ordenadaEstrictamentePorPrimero( $s : seq⟨tupla ⟨\mathbb{Z}, \mathbb{Z}⟩⟩$ ) {  
    ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |s| - 1 \rightarrow_L s[i]_0 < s[i + 1]_0$ ) }  
}
```



Ejercicio 18.a

reemplazarParesPorSiguiete. Dado una secuencia de enteros s , reemplazar los elementos pares de la secuencia por el siguiente



Ejercicio 18.a

reemplazarParesPorSiguiente. Dado una secuencia de enteros s , reemplazar los elementos pares de la secuencia por el siguiente

*Este vamos a
pensarlo juntos*

No te emociones
tanto



Ejercicio 18.a

```
proc reemplazarParesPorSiguiete(inout s : seq⟨ℤ⟩) {  
  requiere { True }  
  asegura {  
    ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |s| \wedge_L \text{divide}(2, s[i]) \rightarrow_L \text{setAt}(s, i, s[i] + 1)$ ) }  
}
```



Pero acá no estamos
programando...

Ejercicio 18.a

```
proc reemplazarParesPorSiguiete(inout s : seq⟨ℤ⟩) {  
  requiere { s = S0 }  
  asegura {  
    (∀i : ℤ) (0 ≤ i < |s| ∧L divide(2, s[i]) →L  
      s = setAt(s, i, S0[i] + 1))  
  }  
}
```

¿Qué problema ven
acá?



Ejercicio 18.a

```
proc reemplazarParesPorSiguiete(inout s : seq⟨ℤ⟩) {  
  requiere { s = S0 }  
  asegura {  
    |s| = |S0| ∧L  
    (∀i : ℤ) (0 ≤ i < |S0| ∧L divide(2, S0[i]) →L s[i] = S0[i] + 1)  
  }  
}
```

A mí no me engañas,
sé que esta sigue
estando mal...



Ejercicio 18.a

```
proc reemplazarParesPorSiguiete(inout s : seq⟨ℤ⟩) {  
  requiere { s = S0 }  
  asegura {  
    |s| = |S0| ∧L  
    (∀ i : ℤ) (0 ≤ i < |S0| ∧L divide(2, S0[i]) →L s[i] = S0[i] + 1)  
    ∧  
    (∀ i : ℤ) (0 ≤ i < |S0| ∧L ¬divide(2, S0[i]) →L s[i] = S0[i])  
  }  
}
```



Ejercicio 20

1. Especificar proc *sacarImpares*(inout $c : \text{conj}(\mathbb{Z})$) que saca los elementos impares del conjunto c
2. Especificar proc *devolverYEliminarMasChico*(inout $c : \text{conj}(\mathbb{Z})$): $\mathbb{Z}$ que recibe un conjunto no vacío, elimina su menor elemento y lo devuelve

No te emociones
tanto



Ejercicio 20

```
proc sacarImpares(inout c : conj⟨ℤ⟩) {  
  requiere { c = C0 }  
  asegura {  
    (∀ e : ℤ) (e ∈ c ↔ (e ∈ C0 ∧ esPar(e)))  
  }  
}
```

```
proc devolverYEliminarMasChico(inout c : conj⟨ℤ⟩) : ℤ {  
  requiere { c = C0 ∧ |c| > 0 }  
  asegura {  
    (∃ m : ℤ) (m ∈ C0 ∧ (∀ e : ℤ) (e ∈ C0 → m ≤ e)) ∧  
    c = C0 - {m} ∧ res = m  
  }  
}
```



Ejercicio 26

Se desea especificar el problema **ordenarYBuscarMayor** que dada una secuencia s de enteros (que puede tener repetidos), ordena dicha secuencia en forma creciente por valor absoluto y devuelve el valor del máximo elemento. Por ejemplo,

- $\text{ordenarYBuscarMayor}([1, 4, 3, 5, 6, 2, 7]) = [1, 2, 3, 4, 5, 6, 7], 7$
- $\text{ordenarYBuscarMayor}([1, -2, 2, 5, 1, 4, -2, -10]) = [1, 1, -2, -2, 2, 4, 5, -10], 5$
- $\text{ordenarYBuscarMayor}([-10, -3, -7, -9]) = [-3, -7, -9, -10], -3$



Ejercicio 27

Se desea especificar el problema **procesarPrefijos** que dada una secuencia s de palabras y una palabra p , remueve todas las palabras de s que no tengan como prefijo a p y además retorna la longitud de la palabra más larga de s que tiene de prefijo a p . Por ejemplo, dados:

$s = ["casa", "calamar", "banco", "recuperatorio", "aprobar", "cansado"]$

$p = "ca"$

Un posible valor para la secuencia s luego de aplicar **procesarPrefijos**(s , p) puede ser $["casa", "calamar", "cansado"]$ y el valor devuelto será **7**.





NO SE ATRASEN



¿Qué sigue?

- Con la clase de hoy pueden resolver **toda** la guía 2 (y *la extra*)
- La clase que viene vemos **tipos abstractos de datos (TADs)**
- En las siguientes clases vamos a seguir usando cosas de lógica y especificación. Recomendamos que se ocupen de lograr entenderlo para no quedar *rezagados*



¡Terminamos!

¡Hagan consultas!

Gracias por
acompañarnos

